计算机操作系统

第2章 操作系统的接口

指令权限与工作模式

- 处理器能处理的指令：
  - 特权指令：需要特殊权限才可使用的指令，通常涉及系统安全
  - 非特权指令：任何情况下都可使用的指令
- 处理器的控制寄存器中用专门的位标识当前的工作模式：
  - 核心态（管态）：已获得特殊权限的状态，能够执行所有指令
  - 用户态（目态）：只能执行非特权指令
- 事实上，现代处理器常常具有更多层权限和工作模式，例如，从高到低划分为特权级0（管态）、特权级1、特权级2、特权级3（目态）

内存和操作系统的内核

- 处理器处于核心态时可将内存的不同区段分别设置为两种类型：
  - 内核（系统）空间：其中存储的程序将运行于核心态上，从而可以使用所有指令，且可以访问所有内存空间数据
  - 用户空间：其中存储的程序将运行于用户态上，于是只能使用非特权指令，且只能访问用户空间数据
- 操作系统通常在启动时指定内核空间，并将其核心程序放置在内核空间内，操作系统的这一部分称作内核，操作系统的其他部分称作核外部分。

操作系统的内核

- 操作系统的内核通常包含以下程序和数据：
  - 与硬件密切相关的操作
  - 关键数据结构
  - 基本中断处理程序
  - 使用频繁的功能模块
- 操作系统的内核具有两个基本特点：
  - 常驻内存
  - 运行在核心态

固件

- 操作系统的启动依赖于一组称作系统固件（Firmware）的特殊软件。
- 固件的程序和数据存储在一个或几个只读存储器（ROM）芯片中，也可将部分程序或数据存储在硬盘中。
- 与PC启动相关的固件有BIOS和UEFI，它们在计算机启动时首先被运行，并进行基本硬件资源的检查，装入操作系统引导程序，提供基本硬件资源的驱动程序和中断处理程序等。

基本输入输出系统(BIOS)

- BIOS包含三个部分
  - POST自检程序：计算机启动后识别硬件配置、自检和初始化
  - 基本启动程序：在存储介质中查找有效的（主）引导程序，若找到，则将其读入内存[0000:7C00]处并开始执行，从而装载引导操作系统启动。
  - 基本硬件驱动程序及其中断处理程序：用于在未加载操作系统时识别和控制基本IO设备（键盘、鼠标、显卡、硬盘等）

硬盘的主引导记录(MBR)

- 一块硬盘可以有最多4个分区，但其中可以有1个扩展分区，扩展分区可以利用类似MBR的记录表进一步建立4个逻辑分区，而其中若存在逻辑扩展分区，又可以进一步分区，因此一块硬盘可以划分出任意多的逻辑分区。

表2-1 MBR结构

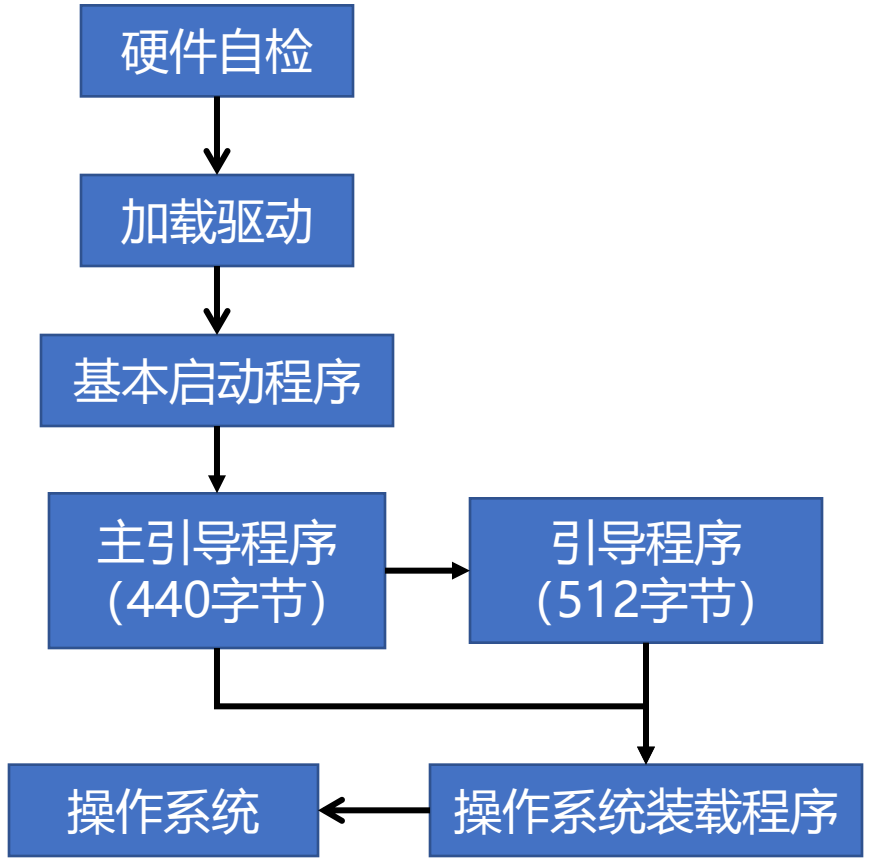
偏移量	字节数	内容
0	440	主引导程序(Master Booter)
440	4	磁盘签名(signature)
444	2	[未定义]
446	16	第1个分区表DPT1
462	16	第2个分区表DPT2
478	16	第3个分区表DPT3
494	16	第4个分区表DPT4
510	2	结束标志符(BRID)：55h AAh(Magic Number)

基于BIOS的操作系统启动过程

- 操作系统装载程序（Loader）

  的作用：

  - 进一步检查和配置系统硬件
  - 建立内核空间，装入操作系统内核，并初始化数据
  - 加载用户登录程序和人机交互环境

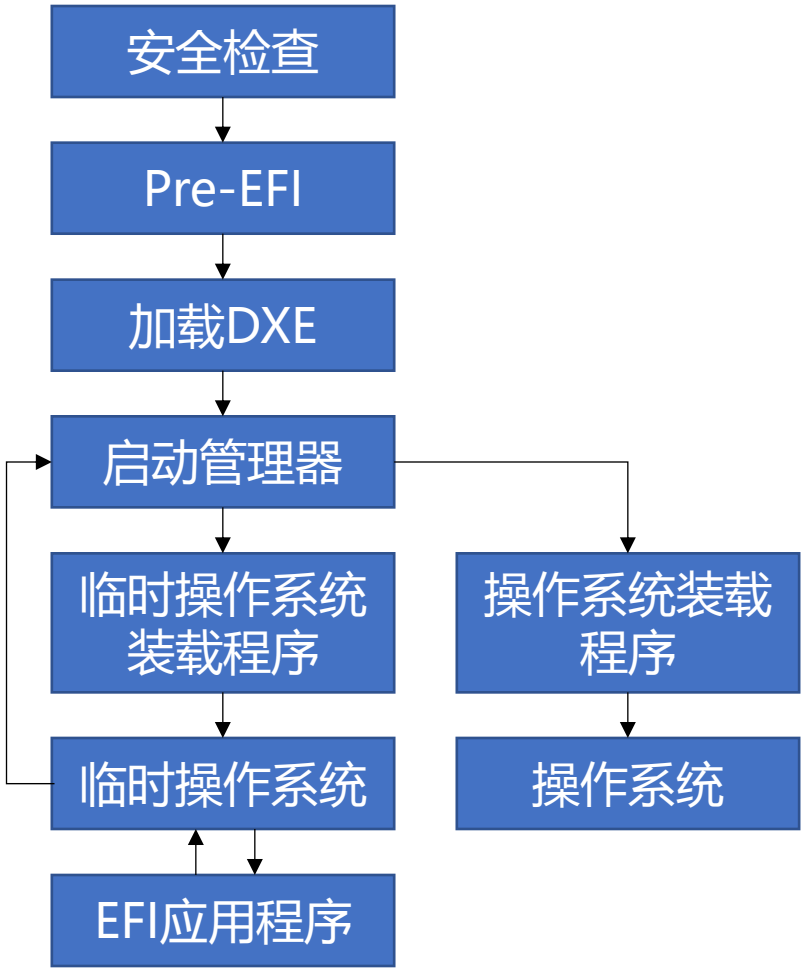


统一的可扩展固件接口(UEFI)

- 英特尔(Intel)公司在1997年为其新推出的高性能处理器，计划设计一种可扩展的、标准化的固件接口规范EFI(Extensible Firmware Interface)，用于计算机系统的启动以及提供与操作系统的接口。在2006年EFI发展成为UEFI（Unified EFI）。
- UEFI的特点：
  - 驱动程序运行环境(DXE)
  - 磁盘管理采用GUID分区方法
  - UEFI应用程序

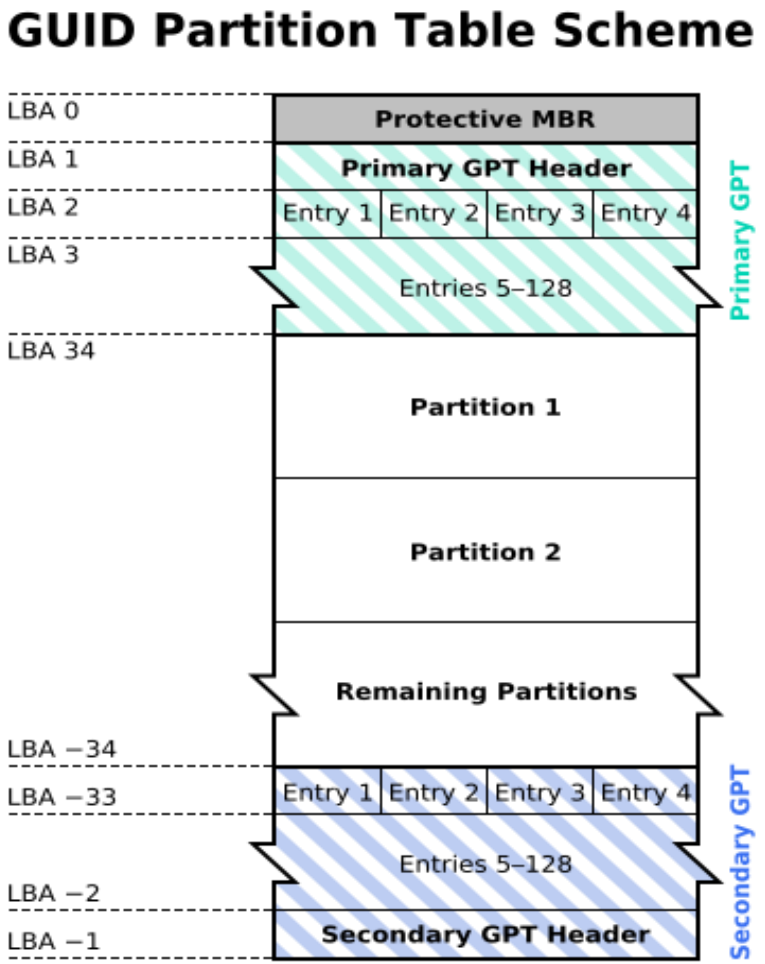
基于UEFI的操作系统启动过程

- UEFI体系包含两部分内容：
  - 平台初始化框架：包含了支持系统启动的底层程序，包括：安全检查、硬件自检和初始化、DXE、启动管理器 等。
  - UEFI映像（UEFI Image）：一组在框架的支持下可执行的程序，包括启动管理器、操作系统装载程序、临时操作系统装载程序、EFI应用程序、设备驱动程序 等。



GUID硬盘分区方式

- UEFI采用GUID硬盘分区方式，优点是：
  - 扇区块号为64位，消除2TB限制
  - 磁盘分区数不受4个限制，最多可划分128个分区
  - 分区扇区数为64位，消除2TB限制
  - 自带备份，增加可靠性
  - 采用CRC32校验，增加完整性
  - 每个分区都有唯一标识GUID



GUID硬盘分区方式

GUID分区表（GPT）的表头（HDR）结构		
起始字节	长度	内容
0	8字节	签名（"EFI PART", 45 46 49 20 50 41 52 54)
8	4字节	版本号
12	4字节	分区表头的大小（单位是字节，通常是92)
16	4字节	分区表头（第0 - 91字节）的CRC32校验
20	4字节	保留，必须是 0
24	8字节	当前LBA（这个分区表头的位置）
32	8字节	备份LBA（另一个分区表头的位置）
40	8字节	第一个可用于分区的LBA（主分区表的最后一个LBA + 1）
48	8字节	最后一个可用于分区的LBA（备份分区表的第一个LBA − 1）
56	16字节	硬盘GUID
72	8字节	分区入口表的起始LBA
80	4字节	分区入口表的表项数量（分区个数）
84	4字节	一个分区入口表项的大小（单位是字节，通常是128）
88	4字节	分区入口表的CRC32校验
92	*	保留，剩余的字节必须是0

操作系统的用户接口--命令接口

- 命令接口是人机交互的主要途径。
- 按命令接口在操作系统内的实现方式，分为
  - 内部命令：内核完成的命令
  - 外部命令：由其他应用程序完成的命令
- 按命令接口的使用方式，分为：
  - 脱机命令：预先编写好，送交执行后无法立即反馈的命令接口形式。例如批处理系统中的JCL。
  - 联机命令：交互式执行，能够立即得到反馈的命令接口形式。例如分时系统中的交互式命令接口。又可细分为字符命令、菜单命令和图标命令。

操作系统的用户接口—程序接口(系统调用)

- 程序接口是程序员在编写应用程序时使用操作系统功能的接口。目前，程序接口通常通过系统调用（System Call）方式实现。
- 系统调用在不同的上下文中，可能表示两种含义：
  - 一组由操作系统设计者编写的内核中的子程序，用以封装用户（程序员）可能使用到的操作系统功能。
  - 程序员在编程中对这些子程序进行调用的行为。
- 系统调用所涉及的子程序属于操作系统内核的一部分，保存于内核空间中，应用程序不能直接访问，于是需要通过特殊方式才能够实现系统调用。

操作系统的用户接口—程序接口(系统调用)

❖ 系统调用的实现：

1. 放置调用号标识

2. 访管指令：发出表示系统调用请求的中断（或陷阱）

3. 处理器保护程序现场，转向中断处理（内核态）

4. 中断处理程序识别中断为系统调用请求，于是转向内核的访管指令处理程序

5. 访管指令处理程序在地址入口表中查找所请求的调用号对应的内核子程序的入口地址

6. 从查找到的地址开始执行内核子程序（系统调用）并得到结果

7. 回复应用程序现场，返回用户态

操作系统的用户接口—程序接口(系统调用)

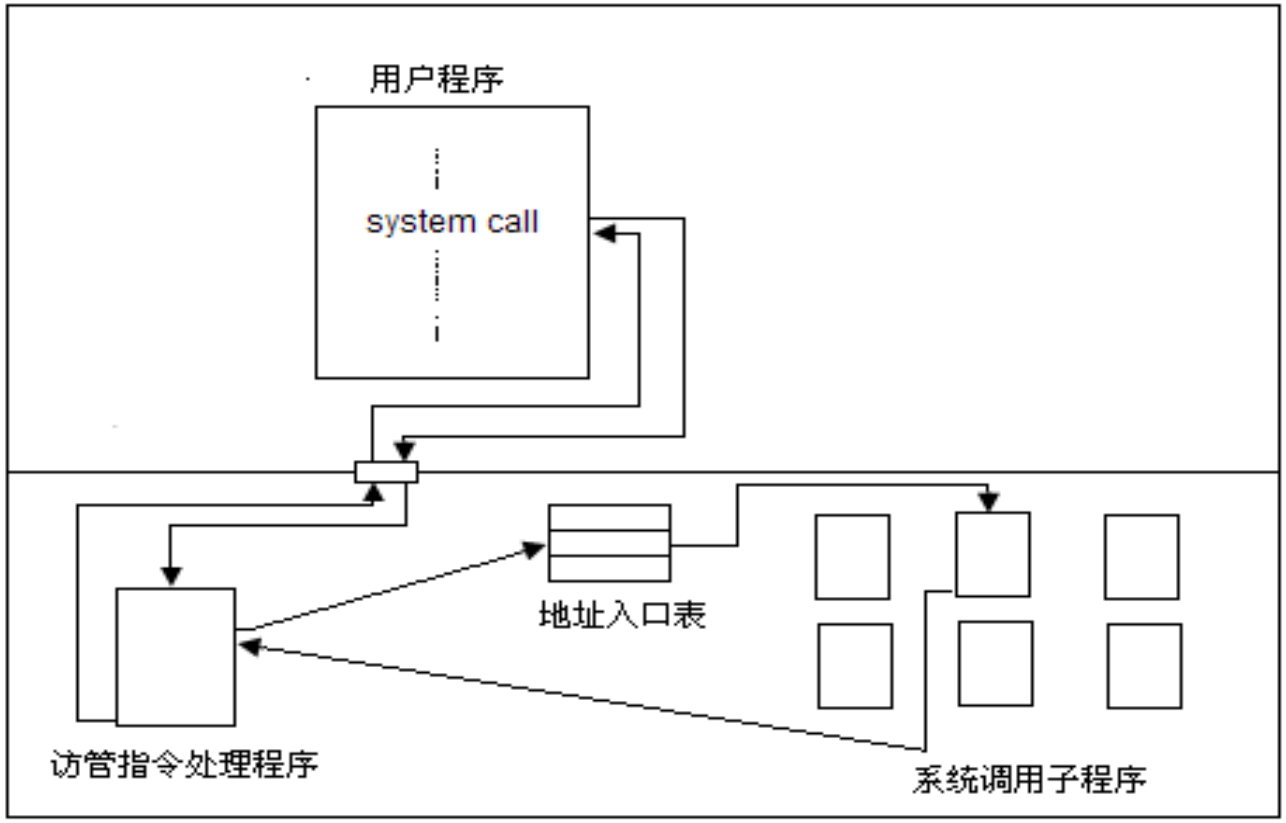


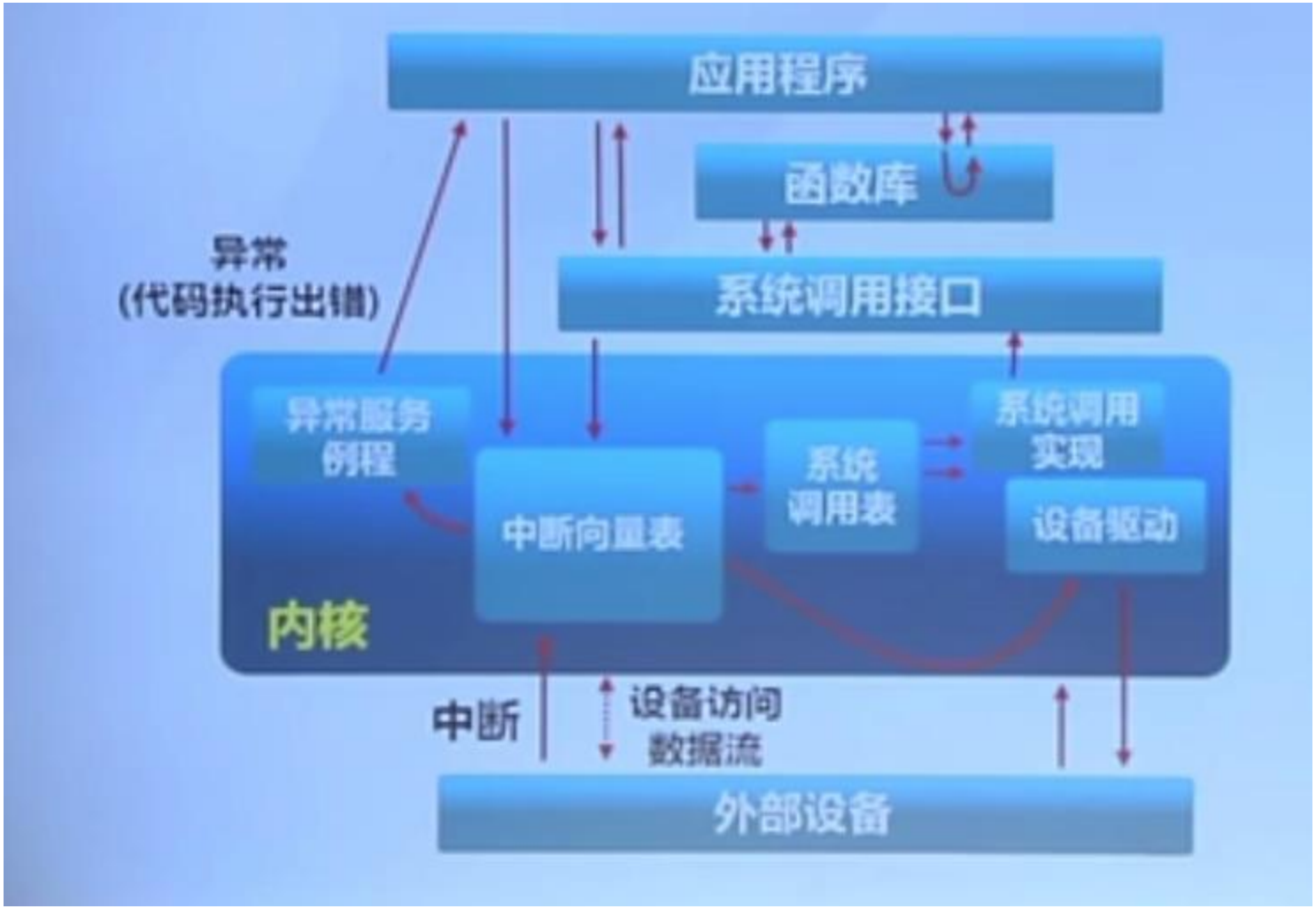
图2-6 系统调用实现过程

操作系统的用户接口—程序接口(系统调用)

❖ 系统调用与用户子程序的对比

对比项目	系统调用	用户子程序
运行环境	内核态（管态）	用户态（目态）
访问方式	访管指令（中断或陷阱）	直接跳转、返回
与主程序关系	与主程序分开、独立	同一进程地址空间
共享性	可在多个进程间共享	同一进程内部调用

内核与应用程序/外部设备如何打交道?



Linux的结构

